Canvas Line Sender 코드
초상세 해설서

Dart · Flutter 문법부터
그리기 → 좌표 변환 → JSON 전송까지
중학생도 따라올 수 있게 아주 자세히 설명

대상 파일	main_android.dart / Canvas Line Sender 앱
설명 수준	입문자~초급자, 코드 독해용
핵심 목표	“이 줄이 왜 있는지”와 “전체 흐름이 어떻게 이어지는지” 이해
읽는 방법	처음엔 1~4 장, 그다음 5 장 이후를 코드와 함께 보기

이 문서는 이전 요약본보다 훨씬 더 자세하게, 변수·함수·이벤트 흐름·좌표계·JSON 구조·UI 구성·CustomPainter 동작까지 단계적으로 풀어쓴 버전이다.

1. 이 앱을 한 문장으로 말하면?

사용자가 화면 위에서 선이나 자유곡선을 그리고, 그 선을 일정 간격의 점들로 잘게 나눈 뒤, 각 점을 이미지 좌표와 mm 좌표로 바꾸고, Z 값까지 붙여서 JSON 형식으로 TCP 서버에 보내는 앱이다.

조금 더 쉽게 말하면, "손으로 경로를 그리면 로봇이나 다른 프로그램이 이해할 수 있는 좌표 리스트로 바꿔 주는 도구"라고 볼 수 있다.

- 그림판처럼 선을 그릴 수 있다.
- 그린 선을 10px 간격 같은 규칙적인 점들로 다시 만든다.
- 화면 좌표를 실제 좌표(mm)로 바꾼다.
- 각 점마다 Z 값을 붙일 수 있다.
- 마지막에 {"points": [[x, y, z], ...]} 형태 JSON 으로 보낸다.

2. 전체 구조를 먼저 큰 그림으로 보기

코드를 처음 읽을 때 가장 흔한 실수는 위에서부터 한 줄씩만 보는 것이다. 이 파일은 기능이 많아서, 먼저 "어떤 덩어리들로 나뉘는지"를 잡아야 덜 헷갈린다.

구역	대표 요소	무슨 역할?
앱 시작	main, CanvasApp	Flutter 앱을 띄운다
상태 보관	_CanvasHomePageState 변수들	현재 선, 점, 선택 상태, 소켓 상태 등을 저장
입력 처리	_onPointerDown/Move/Up	마우스 입력을 받아 그리기, 선택, 이동, 편집 수행
좌표 계산	_rebuildPoints, _canvasToImageClamped, _applyTransform	그림을 점으로 만들고 좌표계를 바꾼다
전송	_ensureConnected, _buildPayload, _sendJson	TCP 연결 후 JSON 전송
화면 표시	build, _buildCanvasArea, _buildControlsPanel, LinePainter	캔버스와 버튼, 표를 화면에 그린다

3. Dart 문법 기초: 이 코드에서 꼭 알아야 할 것

import

다른 라이브러리의 기능을 가져온다. Python 의 import 와 거의 비슷하다. 예를 들어 dart:io 는 파일과 소켓, package:flutter/material.dart 는 버튼·텍스트필드·Scaffold 같은 UI 위젯을 쓸 수 있게 한다.

class

관련된 데이터와 함수를 하나로 묶는 설계도다. CanvasApp, CanvasHomePage, Stroke 같은 것들이 class 다.

extends

기존 클래스를 상속받는다. 예: StatelessWidget, StatefulWidget, CustomPainter. Flutter 에서는 "화면에 보이는 부품"을 만들 때 자주 쓴다.

const

한 번 정해지면 바뀌지 않는 값이다. 실행 중 계속 안 바뀌는 설정값이나 위젯 생성에 자주 쓴다.

final

딱 한 번만 대입 가능한 변수다. const 처럼 컴파일 시점 상수는 아니지만, 한 번 들어간 뒤에는 못 바꾼다.

?

null 이 될 수도 있다는 뜻이다. 예: Offset? _p0; 는 시작점이 아직 없을 수도 있다는 의미다.

!

"여긴 null 이 아니라고 내가 확신한다"는 표시다. _p0!처럼 사용하면 실제 값처럼 접근한다. 대신 정말 null 이면 오류가 난다.

async / await

시간이 걸리는 작업을 기다릴 때 쓴다. 이미지 읽기, 소켓 연결, URL 열기 같은 작업에서 중요하다.

setState

상태가 바뀌었으니 화면도 다시 그리라고 Flutter 에 알려주는 함수다. 이게 없으면 값은 바뀌어도 화면이 안 변할 수 있다.

List<Offset>

Offset 객체들을 담는 리스트다. 점들의 모음이라고 보면 된다.

map / toList

리스트의 각 원소를 다른 형태로 바꿀 때 쓴다. 예: 캔버스 좌표 리스트를 이미지 좌표 리스트로 변환.

4. Flutter 문법 기초: 위젯 트리와 StatefulWidget

Flutter 화면은 "위젯 트리"로 이루어진다. 버튼도 위젯, 글자도 위젯, 화면 전체 틀도 위젯이다.

```
void main() {
  runApp(const CanvasApp());
}
```

main 은 프로그램 시작점이다. runApp 은 "이 위젯을 앱의 시작 화면으로 써라"라고 Flutter 에 말한다.

```
class CanvasApp extends StatelessWidget {
  const CanvasApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
```

```
    title: 'Canvas Line Sender',
    theme: ThemeData(...),
    home: const CanvasHomePage(),
  );
 }
}
```

CanvasApp 은 껍데기 역할이다. 앱 제목, 테마, 첫 화면을 정한다. 실제 작업은 CanvasHomePage 에서 한다.

```
class CanvasHomePage extends StatefulWidget {
 const CanvasHomePage({super.key});

 @override
 State<CanvasHomePage> createState() => _CanvasHomePageState();
}
```

StatefulWidget 은 화면이 계속 변할 때 사용한다. 여기서는 그리기 상태, 선택 상태, 포인트 수, Z 값, 소켓 연결 상태가 계속 바뀌므로 StatefulWidget 이 꼭 필요하다.

createState 는 실제 데이터를 들고 있는 _CanvasHomePageState 객체를 만든다. 화면의 "두뇌"가 이쪽에 있다고 생각하면 된다.

5. 상태 변수 전체 지도

이 파일이 길어 보이는 가장 큰 이유는 상태 변수가 많기 때문이다. 하지만 묶어서 보면 어렵지 않다.

묶음	대표 변수	핵심 의미
기본 설정값	_defaultStepPx, _defaultMmPerPx, _defaultHost, _defaultPort	앱이 처음 켜졌을 때 쓰는 기본값이다.
현재 그리고 있는 도형	_p0, _p1, _rawPoints, _isDrawing	마우스를 누르고 움직이는 동안만 필요한 임시 데이터다.
현재 샘플 점 결과	_pointsPx, _pointsImage, _pointsMm, _deltasMm	그린 결과를 다시 계산한 점들이다. 좌표계별로 따로 저장한다.
현재 Z 값	_currentZValues	현재 미완성 도형이나 stroke 없는 상태에서 쓸 Z 목록이다.
완성된 stroke 들	_strokes	확정된 선/곡선들의 모음이다.
선택/이동/이어그리기	_selectedStrokeIndex, _dragStrokeIndex, _appendPen	어느 stroke 가 선택됐는지, 이동 중인지, 이전 선에 이어 그릴지 기억한다.
선택 박스	_selectMode, _isSelecting, _selectStart, _selectRect	드래그 사각형으로 stroke 를 고를 때 사용한다.
점 편집/스무딩	_isEditingPoint, _editStrokeIndex, _smoothingWindow	샘플 점 직접 이동, 이동 평균 스무딩에 필요하다.
배경 이미지/프레임	_imageFile, _imageSize,	이미지 배경과 프레임 크기, 실제

	_userFrameMmSize, _frameRect	표시 위치를 관리한다.
입력창 컨트롤러	_hostController 등	TextField 에 적힌 글자를 읽고 보관한다.
소켓 상태	_socket, _connectedHost, _connectedPort, _status	현재 서버 연결과 상태 문구를 관리한다.

6. 기본 설정값 자세히

_defaultStepPx

점 간격 기본값이다. 선을 얼마나 촘촘하게 점으로 쪼갤지 정한다. 값이 작을수록 점이 많아지고, 값이 크면 점이 적어진다.

_defaultMmPerPx

1 픽셀이 몇 mm 인지 나타내는 비율이다. 나중에 화면 좌표를 실제 단위로 바꾸는 기준이 된다.

_defaultHost / _defaultPort

TCP 서버 연결 기본 주소다. 아무것도 입력하지 않으면 이 값으로 테스트한다.

7. 현재 그리는 도형 관련 변수

_p0

line, circle, rect 모드에서 시작점 역할을 한다. 예를 들어 line 모드에서는 선의 시작점이고, circle 에서는 중심점처럼 사용된다.

_p1

현재 끝점 또는 마지막 점이다. 마우스를 움직일 때 계속 업데이트된다.

_rawPoints

pen 모드에서 손이 지나간 원본 점들이다. 이 리스트는 울퉁불퉁할 수 있다. 나중에 _resamplePolyline 으로 일정 간격 재샘플링된다.

_isDrawing

현재 마우스를 누른 상태로 그리고 있는지 나타낸다. 그리기 중일 때만 move 이벤트가 도형 생성에 사용된다.

8. 좌표 결과 리스트 4 종 세트

_pointsPx

캔버스 좌표 기준 샘플 점들이다. 화면에 빨간 점으로 찍히는 대상과 가깝다.

_pointsImage

이미지 내부 좌표 기준 점들이다. 프레임 안에서 (0,0)부터 시작하는 좌표로 생각하면 된다.

_pointsMm

실제 mm 단위 좌표다. 최종 전송 시 가장 중요하게 쓰이는 값이다.

_deltasMm

연속 점 사이 차이값이다. 현재 표에는 크게 안 보이지만, 경로 이동량 분석에 유용하다.

9. Z 값 관리가 왜 필요한가?

_currentZValues

x, y 만 보내는 것이 아니라 x, y, z 를 보내기 때문에 필요하다. 현재 그리고 있는 점들 수와 길이가 맞아야 한다.

stroke.zValues

완성된 각 stroke 도 자기만의 Z 목록을 가진다. 샘플 점 개수와 길이가 같아야 인덱스로 정확히 대응된다.

_syncZList

리스트 길이가 모자라면 0.0 을 채우고, 길면 자른다. 이 함수가 없으면 점 수와 Z 수가 안 맞아서 오류가 나기 쉽다.

10. initState 와 dispose: 시작할 때 / 끝날 때 정리

initState 는 이 화면이 처음 생길 때 딱 한 번 호출된다. 여기서는 A2 크기(594 x 420)를 기본 프레임으로 넣고, mm/px 를 1.0 으로 시작한다.

즉, 앱이 켜지자마자 "아직 이미지는 없어도 작업할 기준 프레임은 존재하는 상태"가 된다.

dispose 는 화면이 사라질 때 마지막으로 정리하는 곳이다. TextEditingController 를 해제하고, 열려 있던 소켓도 destroy 로 끊어 준다. 이것을 안 하면 메모리 누수나 연결 잔여 문제가 생길 수 있다.

11. 사용자가 입력한 값 적용 함수들

_applyMmPerPx

텍스트필드에서 사용자가 입력한 값을 double 로 읽어 mm/px 로 저장한다. 0 이하이거나 숫자가 아니면 상태 문구에 invalid 를 띄운다.

_applyStepPx

점 간격(stepPx)을 바꾸는 함수다. 이미 그리고 있는 도형이 있으면 _rebuildPoints 를 바로 호출해서 화면 점 개수도 즉시 다시 계산한다.

_applyZToAll

선택된 stroke 가 있으면 그 stroke 에만, 없으면 전체 stroke 또는 current points 에 Z 를 한꺼번에 같은 값으로 넣는다.

_applyBackgroundSize

사용자가 실제 배경 크기(mm)를 입력하면 프레임 기준을 그 크기로 맞춘다. 여기서는 mm 크기를 px 기준으로 동일하게 저장하고, 확대/축소는 _userFrameScale 로 따로 처리한다.

_setPresetSize

A4, A2, B4 같은 버튼을 눌렀을 때 TextField 값을 채운 다음 _applyBackgroundSize 를 호출한다.

12. 입력 이벤트 3 개가 사실상 이 앱의 심장

사용자가 마우스로 무엇을 하느냐에 따라 앱이 거의 다 결정된다. 그래서 _onPointerDown, _onPointerMove, _onPointerUp 세 함수가 가장 중요하다.

_onPointerDown

- 오른쪽 클릭이면 move 모드 시작
- pen + Shift 이면 샘플 점 편집 시작
- Select 모드면 선택 박스 시작
- 그 외에는 현재 모드(line/pen/circle/rect)에 맞춰 그리기 시작

_onPointerMove

- 이동 중이면 stroke 전체를 끌고 간다
- 점 편집 중이면 선택한 점만 옮긴다
- 선택 박스 중이면 사각형 크기를 갱신한다
- 그리기 중이면 p1 또는 rawPoints 를 갱신하고 _rebuildPoints 실행

_onPointerUp

- 이동 중이면 종료
- 점 편집 중이면 종료
- 선택 중이면 박스로 stroke 선택
- 그리기 중이면 최종 점 계산 후 stroke 리스트에 저장

13. line / pen / circle / rect 모드별 동작 차이

모드	입력 방식	샘플링 방식	결과 느낌
line	시작점과 끝점	두 점 사이를 균일 분할	직선
pen	손이 지나간 rawPoints	polyline 재샘플링	자유곡선
circle	중심과 반지름용 점	원 둘레를 일정 간격 분할	원형 경로
rect	대각선 두 점	사각형 테두리를 선분별 샘플링	직사각형 경로

중요한 점은, pen 모드라고 해서 rawPoints 가 그대로 최종 점이 되는 게 아니라는 것이다. 손이 흔들리며 들어온 원본을 일정 간격 점들로 다시 만드는 과정이 있다.

14. 왜 _rebuildPoints 가 핵심 함수인가?

이 함수는 "현재 사용자가 만든 도형"을 전송 가능한 점 목록으로 바꾸는 공장 같은 역할을 한다. 앱 안에서 보이는 경로와 실제 데이터가 연결되는 순간이 바로 여기다.

- line: p0 와 p1 을 이미지 좌표로 바꾼 뒤, 길이에 따라 점 개수 n 을 계산하고 직선 보간한다.
- circle: 중심과 바깥 점으로 반지름을 구하고, 둘레 길이에 따라 분할 개수를 정한다.
- rect: 사각형 네 변을 각각 샘플링해서 한 바퀴 경로로 만든다.
- pen: rawPoints 를 이미지 좌표로 바꾸고, _resamplePolyline 으로 일정 간격 점 리스트를 만든다.
- 마지막에는 _pointsPx, _pointsImage, _pointsMm, _deltasMm, _currentZValues 까지 모두 동기화한다.

그래서 점 간격을 바꾸거나, 마우스를 움직이거나, 모드를 바꾸거나, 편집을 해도 결국 이 함수가 다시 불리면 현재 데이터가 최신 상태로 맞춰진다.

15. 좌표 변환 3 단계: 캔버스 → 이미지 → mm

이 코드에서 가장 헷갈리기 쉬운 부분이 좌표계다. 좌표계를 단계별로 나눠 생각하면 쉽다.

좌표계	예시	의미
캔버스 좌표	(420, 220)	실제 앱 창 안에서의 위치. 프레임 왼쪽 위가 0,0 은 아님.
이미지 좌표	(100, 50)	프레임 내부 기준 좌표. 왼쪽 위를 0,0 으로 하는 더 논리적인 좌표.
mm 좌표	(100.0, 50.0)	실제 단위로 환산된 값. 로봇/외부 시스템에 보내기 좋음.

_canvasToImageClamped

사용자가 찍은 화면 좌표를 프레임 내부 좌표로 바꾸고, 프레임 밖으로 나가지 않게 clamp 한다. 즉, 사용자가 살짝 바깥을 찍어도 좌표가 프레임 범위 안에 붙잡힌다.

_imageToCanvas

반대로 이미지 내부 좌표를 다시 화면에 그릴 수 있는 캔버스 좌표로 바꾼다. 샘플 점들을 화면에 찍을 때 필요하다.

_applyTransform

이미지 좌표에 mm/px 스케일을 적용한다. 현재는 yFlip=false, rotationDeg=0.0 이라 단순 스케일링이지만, 나중에 필요하면 Y 축 뒤집기나 회전도 넣을 수 있게 구조가 준비되어 있다.

16. 샘플링 수학 함수들 쉽게 해석하기

_resamplePolyline

울퉁불퉁한 자유곡선 입력을 stepPx 간격으로 다시 자른다. carry 변수를 써서 "지난 선분에서 남은 거리"까지 고려한다. 그래서 선분이 바뀌어도 전체적으로 일정 간격을 유지할 수 있다.

_sampleSegment

한 선분 AB 를 일정 간격 점들로 나눈다. 직선 보간 공식을 사용한다.

_circlePoints

둘레 길이 = 2πr 을 이용해 필요한 점 개수를 정하고, cos/sin 으로 원 위 점을 생성한다.

_rectPoints

사각형 네 모서리를 만든 뒤, 각 변을 _sampleSegment 로 샘플링한다. 중복 모서리는 removeAt(0)로 한 번씩 제거한다.

_buildDeltas

인접 점들 차이값을 만든다. 예를 들어 로봇이 다음 점으로 얼마나 움직여야 하는지 계산할 때 유용하다.

17. stroke 라는 개념을 꼭 이해해야 하는 이유

stroke 는 "한 번 완성된 선 묶음"이다. 사용자가 line 하나를 그리거나 pen 으로 한 번 쭉 그리면 stroke 하나가 생긴다.

- path: 실제 사용자가 그린 경로 느낌을 담는 점들
- samples: 전송용으로 균일 간격 재구성된 점들
- zValues: samples 와 1:1 대응하는 Z 목록
- basePath / baseSamples: 스무딩 전 원본 백업

즉, 현재 그리는 중인 임시 데이터와 달리, stroke 는 "확정 저장된 결과물"이다. 여러 개가 쌓이면 _strokes 리스트 안에 들어간다.

18. Continue 기능은 내부적으로 무엇을 하나?

_appendPen 이 true 면 새로 그리기 시작할 때 완전히 새 선을 만드는 것이 아니라, 마지막 stroke 의 끝점에서 이어 그리도록 한다.

line 모드에서도 마지막 stroke 의 끝점을 시작점으로 잡을 수 있고, pen 모드에서는 _rawPoints 첫 점에 이전 마지막 점을 넣는다.

업 시점(_onPointerUp)에는 path 와 samples 의 첫 점이 이전 stroke 마지막 점과 같으면 중복을 제거하고, 새로 추가된 samples 길이만큼 zValues 도 0.0 으로 늘린다.

이 부분이 매우 중요하다. 점은 이어 붙였는데 Z 목록을 안 늘리면 길이가 어긋나기 때문이다.

19. 선택, 이동, 점 편집, 스무딩

_selectStrokeByRect

선택 박스(Rect)와 겹치는 stroke 중에서, 박스 중심에 가장 가까운 것을 선택한다. 단순히 "겹치기만 하면 첫 번째"가 아니라, 거리까지 계산해 더 그럴듯한 대상을 고른다.

_beginMove / _updateMove / _endMove

오른쪽 클릭 드래그로 stroke 전체를 옮긴다. 마지막 마우스 위치와 현재 위치 차이(delta)를 stroke.path 와 stroke.samples 모두에 더한다.

_beginEditPoint / _updateEditPoint / _endEditPoint

Shift+펜 모드에서 가장 가까운 샘플 점을 골라 직접 이동한다. samples 뿐 아니라 path 의 가까운 점도 같이 고친다.

_applySmoothingToSelected

이동 평균으로 선을 부드럽게 만든다. slider 값은 항상 홀수 창 크기로 유지된다. 원본(basePath/baseSamples)을 보관해 두고, 슬라이더 값을 바꿀 때마다 원본 기준으로 다시 계산한다.

20. 히트 테스트는 어떻게 클릭 대상을 찾나?

컴퓨터는 "사람이 선을 눌렀다"는 감각이 없다. 그래서 수학으로 판단해야 한다.

- _hitTestStroke: 클릭 위치와 각 stroke.path 사이 최소 거리를 계산해 threshold 안이면 선택
- _minDistanceToPath: 폴리라인 전체를 이루는 모든 선분과의 거리를 검사
- _distanceToSegment: 점과 선분 사이 최소 거리 계산
- _hitTestPoint: 각 sample 점과 클릭 지점 사이 거리를 직접 비교

즉, 선을 누른다는 것은 "내 클릭 위치가 이 선에 충분히 가깝다"로 바꿔 계산하는 것이다.

21. 네트워크 보조 함수들

_networkDebugInfo

현재 장치의 네트워크 인터페이스와 IP 주소들을 문자열로 모은다. 연결 실패 때 진단용으로 상태 문구에 함께 보여준다.

_tryParseIpv4 / _same24Subnet

문자열 IP 를 숫자 4 개로 나누고, 앞의 세 바이트가 같은지 비교한다. 보통 같은 /24 대역인지 간단히 보는 용도다.

_selectSourceAddress

장치에 여러 네트워크가 있을 때, 목표 host 와 같은 대역의 로컬 IP 를 우선 sourceAddress 로 고르려는 함수다. 예를 들어 Wi-Fi 와 이더넷이 동시에 있을 때 더 안정적으로 연결되도록 도와준다.

22. TCP 연결 과정 아주 자세히

_ensureConnected 는 전송 전 반드시 호출되는 안전장치다.

- host 와 port 가 유효한지 검사한다.
- 이미 같은 host/port 에 연결된 소켓이 있으면 재사용한다.
- 다른 서버에 연결된 소켓이 남아 있으면 destroy 로 끊는다.

- sourceAddress 를 골라 Socket.connect 를 시도한다.
- listen 을 걸어 서버 응답이 오면 상태 문구를 업데이트한다.
- onError, onDone 처리로 에러와 연결 종료를 반영한다.

즉, 이 함수는 단순히 "연결해"가 아니라 "입력 검증 → 기존 연결 정리 → 새 연결 → 응답 감시"까지 담당한다.

23. JSON payload 만들기

_buildPayload 는 최종 전송 데이터 구조를 조립한다. 여기서 핵심은 Current 만 보낼지, stroke 여러 개를 이어서 보낼지를 나눈다는 점이다.

```
{
  "points": [
    [12.34, 56.78, 0.00],
    [14.11, 58.40, 0.00],
    ["000"]
  ]
}
```

stroke 가 여러 개일 때는 각 stroke 끝마다 ["000"] 마커를 추가한다. 이 마커는 "여기서 한 stroke 끝"이라고 서버 쪽에 알려 주는 구분자다.

각 좌표는 소수점 둘째 자리까지 맞춰 보낸다. double.parse(...toStringAsFixed(2)) 형태를 쓰는 이유는 보기 좋은 문자열이 아니라 실제 숫자 값으로 남기기 위해서다.

24. _sendJson 의 실제 흐름

- 보낼 데이터가 있는지 먼저 검사한다.
- _ensureConnected 로 연결을 확보한다.
- _buildPayload 로 points 구조를 만든다.
- jsonEncode 로 문자열 JSON 으로 바꾼다.
- 소켓에 jsonText + 개행(\n)을 써서 보낸다.
- flush 로 밀어 넣고, 상태 문구를 "몇 점 보냈는지"로 업데이트한다.

개행 문자를 붙이는 이유는 서버가 한 줄 단위 메시지로 읽기 쉽게 하기 위함일 가능성이 크다.

25. build 함수: 화면 배치의 중심

build 는 상태가 바뀔 때마다 다시 호출될 수 있는 함수다. Flutter 는 여기서 현재 상태를 보고 "지금 화면을 어떻게 그릴지" 다시 결정한다.

- MediaQuery 로 화면 방향이 가로인지 세로인지 판단
- _buildCanvasArea 와 _buildControlsPanel 을 각각 생성

- 가로면 Row 로 나란히, 세로면 Column 으로 위아래 배치

즉, 같은 기능이지만 화면 방향에 따라 레이아웃을 자동으로 바꾸는 반응형 구조다.

26. _buildCanvasArea: 실제 그림 영역

캔버스 영역은 단순 빈 공간이 아니라 여러 층(layer)으로 이루어진다.

- 가장 아래: 회색 테두리의 프레임 상자
- 그 위: 업로드한 배경 이미지(Image.file)
- 가장 위: Listener + CustomPaint

LayoutBuilder 로 현재 사용 가능한 크기를 받아 프레임 크기를 계산하고, 그 결과를 _frameRect 에 저장한다. 나중에 좌표 변환 함수들은 이 _frameRect 를 기준으로 삼는다.

즉, _frameRect 는 "화면 속 실제 작업판의 위치와 크기"를 기억하는 매우 중요한 값이다.

27. _buildControlsPanel: 모든 조작이 모이는 곳

- 현재 시작점, 끝점, 길이, 포인트 수, 상태 문구 표시
- Points Table 표시와 Z 편집
- Host / Port 입력
- Reset / Test TCP / Send / Delete / Upload / Onshape 버튼
- Line / Pen / Circle / Rect 모드 전환
- Continue / Select 토글
- Point gap, Smooth, BG Size, Frame Scale, mm/px 설정

이 패널은 사실상 "조종석"이다. 사용자가 값을 바꾸면 대부분 setState 와 연결되고, 관련 함수가 다시 계산을 수행한다.

28. DataTable 과 Z 편집

_buildPointRows 는 표 한 줄 한 줄을 만든다. X 와 Y 는 텍스트로 보여주고, Z 는 TextFormField 로 직접 수정 가능하게 해 둔다.

선택된 stroke 가 있으면 그 stroke 의 좌표와 Z 만 보여주고, 선택이 없으면 전체 stroke 또는 current points 를 보여 준다.

Z 칸을 수정할 때 row.strokeIndex 가 있으면 해당 stroke.zValues 를 고치고, 없으면 _currentZValues 를 고친다. 즉, 표 편집이 곧 실제 데이터 수정이다.

29. 길이 계산 함수

_polylineLength 는 점 리스트 총 길이를 구한다. 인접한 점 사이 거리들을 전부 더한다.

_lengthForMode 는 현재 mode 가 무엇인지에 따라 길이를 다르게 계산한다.

- line: 시작점~끝점 거리
- circle: 둘레 길이 2πr
- rect: 2(w+h)
- pen: 샘플 점 또는 rawImage 점들의 polyline 길이

30. LinePainter: 상태를 실제 화면에 그리는 역할

이 클래스는 데이터를 바꾸지 않는다. 오직 현재 상태를 읽어서 화면에 선과 점과 라벨을 그릴 뿐이다. 그래서 "렌더러"라고 생각하면 된다.

stroke.path 그리기

각 stroke 를 Path 로 만들어 선으로 그린다. 선택된 stroke 는 주황색, 나머지는 파란색 계열로 그린다.

stroke.samples 점 찍기

샘플 점마다 작은 원을 찍는다. 선택된 stroke 의 점은 색을 바꿔 눈에 띄게 한다.

현재 미완성 도형 표시

line, pen, circle, rect 모드에서 아직 확정 전인 도형도 실시간으로 그려 준다.

인덱스 표시

각 점 옆에 번호를 그려서 몇 번째 점인지 보이게 한다.

선택 박스 표시

selectRect 가 있으면 반투명 느낌의 사각형 외곽선을 그린다.

shouldRepaint 는 이전 painter 와 상태가 달라졌는지 보고 다시 그릴지 결정한다. 값이 바뀌었는데 여기서 false 가 되면 화면 갱신이 안 될 수 있다.

31. WebViewerPage 는 왜 따로 뒀을까?

Onshape 같은 웹 사이트를 앱 내부에서 열기 위한 작은 화면이다. Windows 는 webview_windows, 모바일은 webview_flutter 를 쓰도록 분기한다.

플랫폼마다 쓸 수 있는 WebView 구현이 다르기 때문에 initState 에서 분기하고, build 에서도 Platform.isWindows 여부로 서로 다른 위젯을 반환한다.

즉, "웹 사이트 열기"라는 같은 목적을 플랫폼별로 다른 도구로 처리하는 예시다.

32. 이 코드에서 초보자가 자주 헷갈리는 포인트

- _pointsPx 와 _rawPoints 는 같은 것이 아니다. raw 는 원본, pointsPx 는 샘플링 후 결과다.
- _pointsImage 와 _pointsMm 도 다르다. 하나는 프레임 내부 좌표, 하나는 실제 단위 좌표다.

- stroke.path 와 stroke.samples 도 다르다. path 는 경로 느낌, samples 는 전송용 균일 점이다.
- build 안에서 _frameRect 가 매번 다시 계산될 수 있다. 그래서 좌표 변환은 항상 최신 frameRect 를 기준으로 해야 한다.
- setState 를 빼먹으면 값은 바뀌어도 화면이 안 바뀐다.
- TextEditingController 는 dispose 에서 정리해야 한다.
- null 가능 변수(Offset?)는 접근 전에 존재 여부를 꼭 확인해야 한다.

33. 실제 사용 흐름을 예시로 따라가 보기

- 앱을 켠다. 기본 프레임(A2)이 보인다.
- 이미지를 업로드하면 배경으로 깔린다.
- Line 모드에서 마우스로 시작점을 찍고 끝점까지 드래그한다.
- _onPointerDown 이 시작점을 저장하고 _isDrawing=true 가 된다.
- _onPointerMove 때마다 _p1 이 바뀌고 _rebuildPoints 가 호출되어 샘플 점들이 보인다.
- _onPointerUp 시 stroke 가 _strokes 에 저장된다.
- 표에서 Z 를 10 으로 바꾸거나 Apply Z All 을 누른다.
- Send 를 누르면 _ensureConnected → _buildPayload → _sendJson 순으로 실행되어 서버로 전송된다.

34. 유지보수 관점에서 개선하면 좋은 점

현재 코드도 충분히 기능적이지만, 더 커지면 분리하면 좋은 부분들이 보인다.

- 좌표 변환 로직을 별도 helper/service 클래스로 분리
- 네트워크 연결 로직을 socket service 로 분리
- Stroke 편집, 선택, smoothing 관련 기능을 별도 controller 로 분리
- Current drawing state 와 committed stroke state 를 더 명확히 분리
- JSON 포맷을 전용 모델 클래스로 만들기
- 모드별 로직(line/pen/circle/rect)을 strategy 패턴처럼 나누기

하지만 학습용으로는 지금처럼 한 파일에 모여 있는 구조가 오히려 전체 흐름을 한눈에 보기에 좋다는 장점도 있다.

35. 면접·발표에서 이 코드를 설명한다면 이렇게 말하면 된다

“Flutter 로 만든 좌표 입력 도구입니다. 사용자가 캔버스에 선·자유곡선·원·사각형을 그리면, 이를 일정 간격의 점들로 재샘플링하고, 화면 좌표를 이미지 좌표와 실제 mm 좌표로 변환한 뒤, 점별 Z 값과 함께 JSON payload 로 TCP 서버에 전송합니다. 여러 stroke 관리, 선택/이동/점 편집, 스무딩, 배경 이미지 기준 프레임 설정도 지원합니다.”

이 정도로 말하면 기능, 데이터 처리, 네트워크 전송까지 한 번에 전달된다.

36. 핵심 함수 15 개만 다시 압축 정리

함수	한 줄 설명
main	앱 시작
CanvasApp.build	앱 껍데기 생성
_onPointerDown	입력 시작 처리
_onPointerMove	입력 중 갱신
_onPointerUp	입력 종료 및 저장
_rebuildPoints	현재 도형을 샘플 점으로 재계산
_canvasToImageClamped	캔버스→이미지 좌표 변환
_applyTransform	이미지→mm 좌표 변환
_resamplePolyline	자유곡선 재샘플링
_ensureConnected	TCP 연결 확보
_buildPayload	최종 JSON 데이터 생성
_sendJson	실제 전송
_buildCanvasArea	캔버스 화면 구성
_buildControlsPanel	제어 패널 구성
LinePainter.paint	실제 화면 그리기

37. 마지막 요약

이 파일은 단순한 “캔버스 그리기 앱”이 아니다. 입력 처리, 상태 관리, 좌표 변환, 곡선 재샘플링, Z 값 편집, 다중 stroke 관리, 선택/이동/편집, TCP 통신, WebView 보조 기능까지 들어 있는 꽤 종합적인 Flutter 애플리케이션이다.

하지만 큰 흐름은 단순하다.

- 사용자가 그림을 그린다.
- 앱이 그것을 샘플 점들로 바꾼다.
- 점들을 이미지 좌표와 mm 좌표로 바꾼다.
- 필요한 Z 값을 붙인다.
- JSON 으로 서버에 보낸다.

이 다섯 줄만 머릿속에 남겨도 전체 구조를 잃지 않는다. 그리고 세부 사항은 그 다섯 줄 아래에서 각각 어떤 함수가 담당하는지만 연결해서 보면 된다.